Failure (watch it loop, then measure the cost) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `loop_safety_cost_practice.py`: a step limit only feels necessary once you've watched a loop run away. This exercise lets one run — safely, with a deadline — then puts a number on what looping costs compared with one direct call. **If not:** the Build Task's `MaxIterationsExceeded` would be a rule you follow without ever having seen what it protects you from.

Basic Version

Approach 1 — a huge step limit, and a deadline that always stops it

```
# loop_safety_cost_practice.py
import time

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI


@tool
def always_ask_again(city: str) -> str:
    """Look up the weather for a city. Always call it again to check."""
    return "Not sure yet - call this tool again."


model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
model_with_tools = model.bind_tools([always_ask_again])
messages = [{"role": "user", "content": "What's the weather in Paris?"}]

# why: a huge step limit on purpose — but never unlimited: the clock
# below always stops it (Doc02's "never a call without a cap" rule)
max_iterations = 1000
deadline = time.time() + 10

for step in range(max_iterations):
    if time.time() > deadline:
        print(f"Stopped by the 10-second deadline after {step} steps")
```

```
        break
    response = model_with_tools.invoke(messages)
    messages.append(response)
    if not response.tool_calls:
        print("Final answer:", response.content)
        break
    for call in response.tool_calls:
        result = always_ask_again.invoke(call["args"])
        print(f"step {step}: {call['name']} -> {result}")
        messages.append(
            {"role": "tool", "content": result, "tool_call_id": call["id"]}
        )
```

Expected output (the step count depends on how fast the API answers):

```
step 0: always_ask_again -> Not sure yet - call this tool again.
step 1: always_ask_again -> Not sure yet - call this tool again.
...
Stopped by the 10-second deadline after 14 steps
```

The loop really runs away — only the deadline stops it. But nothing measures the cost, and the two limits aren't reusable.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — both limits in one function, and the cost measured

```
# loop_safety_cost_practice.py
import time

from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

from react_loop_practice import MaxIterationsExceeded


@tool
def always_ask_again(city: str) -> str:
    """Look up the weather for a city. Always call it again to check."""
    return "Not sure yet - call this tool again."


def run_with_limits(
    task: str, max_iterations: int, timeout_seconds: float
) -> str:
    # why: two limits, whichever comes first — steps AND a deadline
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    model_with_tools = model.bind_tools([always_ask_again])
    messages = [{"role": "user", "content": task}]
    deadline = time.time() + timeout_seconds
    steps = []

    for step in range(max_iterations):
        if time.time() > deadline:
            raise MaxIterationsExceeded("deadline passed", steps)
        response = model_with_tools.invoke(messages)
        messages.append(response)
        if not response.tool_calls:
            return response.content
        for call in response.tool_calls:
            result = always_ask_again.invoke(call["args"])
            # how: usage_metadata is LangChain's name for Doc04's usage —
            # each step keeps its own token count in the log
            tokens = response.usage_metadata["total_tokens"]
```

```
            steps.append({"step": step, "tool": call["name"], "tokens": tokens})
            messages.append(
                {"role": "tool", "content": result, "tool_call_id": call["id"]}
            )

    message = f"No answer after {max_iterations} steps"
    raise MaxIterationsExceeded(message, steps)


def main() -> None:
    try:
        run_with_limits("What's the weather in Paris?", 5, 30)
    except MaxIterationsExceeded as e:
        loop_tokens = 0
        for step in e.steps:
            loop_tokens = loop_tokens + step["tokens"]
        print(f"case 1 (step limit)  -> stopped after {len(e.steps)} steps")
        print(f"case 1 (tokens used) -> {loop_tokens}")

    model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    direct = model.invoke("What's the capital of France?")
    direct_tokens = direct.usage_metadata["total_tokens"]
    print(f"case 2 (one call)    -> {direct_tokens} tokens")


if __name__ == "__main__":
    main()
```

Expected output (token counts vary by model and prompt):

```
case 1 (step limit)  -> stopped after 5 steps
case 1 (tokens used) -> 1210
case 2 (one call)    -> 21 tokens
```

Difference from Basic: steps and a deadline live in one function that raises the same named error for both, and each step records its tokens, so the partial log shows exactly what the runaway loop cost — many times one direct call.

Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 2 — a timeout around the whole run, using a thread

Story — `run_agent_bounded()`: sometimes you can't change the loop's code. Running the whole `run_agent()` in a second thread (`concurrent.futures`) and waiting at most `timeout_seconds` puts a deadline on it from the outside. **If not:** a loop you don't own could keep a user waiting with no limit at all.

```
# loop_safety_cost_practice.py
import concurrent.futures

from react_loop_practice import MaxIterationsExceeded, run_agent


def run_agent_bounded(task: str, timeout_seconds: float) -> str:
    # how: run the whole loop in a second thread, and stop WAITING for it
    # after timeout_seconds — the loop itself isn't changed at all
    pool = concurrent.futures.ThreadPoolExecutor(max_workers=1)
    future = pool.submit(run_agent, task, 40)
    try:
        return future.result(timeout=timeout_seconds).final_answer
    except concurrent.futures.TimeoutError:
        raise TimeoutError(f"no answer within {timeout_seconds}s")
    finally:
        # why: don't wait for the thread here — but it still runs (and
        # still pays for its calls) until its own loop ends
        pool.shutdown(wait=False)


try:
    print(run_agent_bounded("What's 5 + 7?", 30))
    print(run_agent_bounded("Add 1 + 2, then add 3 to that.", 0.001))
except TimeoutError as e:
    print(f"TimeoutError: {e}")
except MaxIterationsExceeded as e:
    print(f"MaxIterationsExceeded: {e}")
```

Expected output:

```
5 + 7 is 12.
TimeoutError: no answer within 0.001s
```

The catch: you stop *waiting*, but the thread keeps running — and keeps paying for its calls — until its own loop ends. Approach 1's in-loop check stops *before* the next call, so it wastes less.

Back to the exercise · Hint 1 · Hint 2 · Solution

Approach 3 — why the cost grows faster than the step count

Story — the growing `messages` list: every step adds 2 items (the request and its result), and the *whole* list is sent again on every call — so step 5 costs more than step 1. Printing the list's size makes that visible. **If not:** "5 steps cost 5 times one call" would feel true, and be wrong.

```
# loop_safety_cost_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI


@tool
def always_ask_again(city: str) -> str:
    """Look up the weather for a city. Always call it again to check."""
    return "Not sure yet - call this tool again."


model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
model_with_tools = model.bind_tools([always_ask_again])
messages = [{"role": "user", "content": "What's the weather in Paris?"}]
looping_tokens = 0

for step in range(5):
    response = model_with_tools.invoke(messages)
    looping_tokens = looping_tokens + response.usage_metadata["total_tokens"]
    messages.append(response)
    for call in response.tool_calls:
        result = always_ask_again.invoke(call["args"])
        messages.append(
            {"role": "tool", "content": result, "tool_call_id": call["id"]}
        )
    # why: the WHOLE list is sent again on every call — watch it grow
    print(f"step {step}: messages list now has {len(messages)} items")

direct = model.invoke("What's the capital of France?")
direct_tokens = direct.usage_metadata["total_tokens"]
print(f"looping run: 5 steps, {looping_tokens} tokens")
print(f"direct call: 1 step, {direct_tokens} tokens")
# how: :.0f rounds to a whole number — "12x", not "12.3333x"
print(f"ratio: {looping_tokens / direct_tokens:.0f}x")
```

Expected output (token counts vary):

```
step 0: messages list now has 3 items
step 1: messages list now has 5 items
step 2: messages list now has 7 items
step 3: messages list now has 9 items
step 4: messages list now has 11 items
looping run: 5 steps, 1210 tokens
direct call: 1 step, 21 tokens
ratio: 58x
```

Which one should you actually write? Approach 1 in real code — steps and a deadline inside the loop. Approach 2 when you can't change the loop; Approach 3's token logging is worth keeping everywhere. The Build Task's `run_agent()` keeps the step limit, and Project 2 uses the same `MaxIterationsExceeded` with its log.